Lab 6

Start Assignment

- Due Wednesday by 11:59pm
- Points 100
- Submitting a file upload
- Available after Mar 4 at 12am

Overview

In Lab 6, you will plan paths for a Turtlebot operating on the 2D plane.

Similar to Lab 4:

- We will assume that a grid map is given with some grid cells marked as obstacles.
- The obstacles on the grid have been inflated by the radius of the robot.
- The task is to apply RRT* algorithm to find an optimal path from a given start configuration to a given goal configuration without colliding with obstacles.
- Consider the robot as a point (with only x and y coordinates). In this case, the robot is only capable of translation and the discretized C-space is the same as the provided grid map. Hence, the task is to find a path from a given start cell to a given goal cell. During the search, you can treat each 2D cell as a node.

RRT* Algorithm

RRT* is an asymptotically optimal, sampling-based algorithm that finds the shortest path from a start configuration to a goal configuration in a graph or grid. It does this by randomly sampling points from free space and connecting them by checking for collisions. It differs from traditional RRT by periodically “rewiring” the tree to keep the tree optimized for path finding.

Sample Code

- Open a new terminal and go to your `eecs256aw25` package. We will create a new python script.

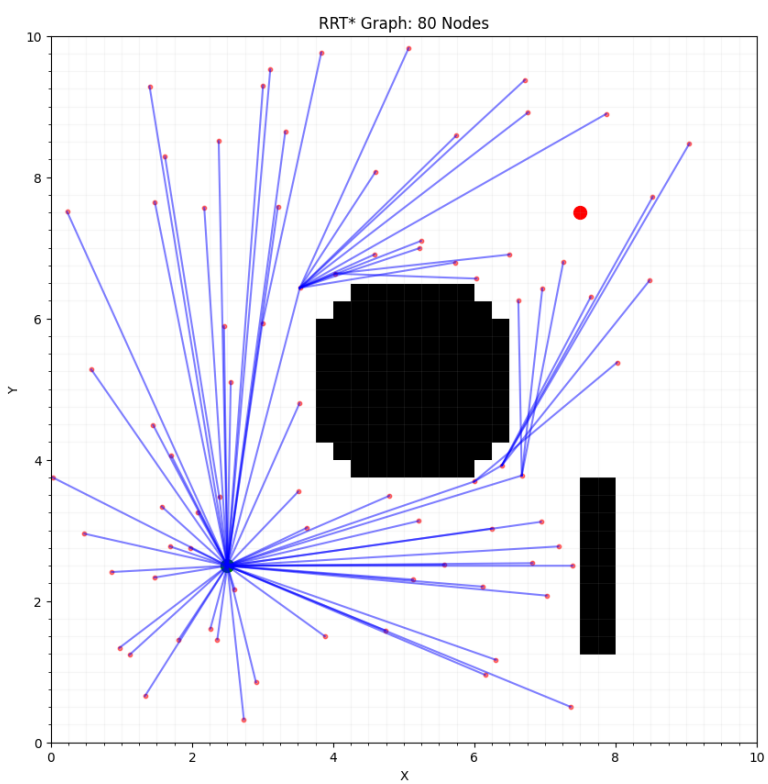
```
import numpy as np
import matplotlib.pyplot as plt
import random
import math

class Node:
    def __init__(self, x, y):
        self.x = x
        self.y = y
        self.parent = None
        self.cost = 0.0
```

```
class RRTStar:
    def __init__(self,
                 start: tuple = (0, 0),
                 goal: tuple = (0, 0),
                 obstacle_grid: np.array = None,
                 grid_resolution: float = 0.25,
                 expand_distance: float = 5.0,
                 connect_circle_dist: float = 20.0):

        self.start = Node(start[0], start[1])           # Start Node in the grid
        self.goal = Node(goal[0], goal[1])              # End Node in the grid
        self.grid = obstacle_grid                       # The obstacle grid to parse
        self.expand_dist = expand_distance              # Maximum distance to grow
        self.connect_circle_dist = connect_circle_dist  #
        self.node_list = [self.start]                   # Graph will definitely contain the start
node
        self.grid_resolution = grid_resolution          # Resolution of the obstacle grid

    def plan(self, N:int = 100):
        """
        Generate an RRT* plan after running N iterations.
        """

        # Generate graph for N iterations
        for iteration in range(N):

            # TODO: Use the helper functions to complete the RRT* plan here
            pass

        # Generate final path
        return

    def steer(self, start_node: Node = None, end_node: Node = None, extend_length=float("inf")) -
> Node:
        """
        Create a node in the direction of the "end" node from the "start" node.
        """
        pass

    def connect(self, new_node, near_inds):
        """
        Connect the tree along a minimum cost path.
        """
        pass

    def rewire_graph(self, new_node, near_inds):
        """
        Rewire the graph.
        """
        pass

    def check_collision(self, n_1: Node = None, n_2: Node = None) -> bool:
        """
        Use Bresenham's Line algorithm to check for obstacles.
        """
        pass

    def get_random_node(self):
        """
        Get a random position from the provided grid.
        """
        pass

    def get_distance(self, n_1: Node = None, n_2: Node = None) -> float:
        """
        Return euclidean distance between 2 nodes.
```

```
        """
        pass

    def get_distance_and_angle(self, n_1: Node = None, n_2: Node = None) -> tuple:
        """
        Compute the euclidean distance and the angle between the 2 nodes.
        """
        pass

    def get_distance_list(self, n: Node = None) -> list:
        """
        Get distance between the provided Node "n" and all the nodes in the graph.
        """
        pass

    def get_near_nodes(self, new_node: Node = None) -> list:
        """
        Get a list of the near nodes in the current graph for a given node.
        """
        pass

    def get_nearest_node_idx(self, random_node: Node = None) -> int:
        """
        Get the index of the nearest node to the "random" node.
        """
        pass

    def plot(self, visualize_nodes: bool = False) -> None:
        """
        Plot the graph.
        """

        plt.figure(figsize=(10, 10))

        # Display obstacle grid
        width = self.grid.shape[1] * self.grid_resolution
        height = self.grid.shape[0] * self.grid_resolution
        plt.imshow(self.grid, cmap='Greys', origin='lower',
                extent=[0, width, 0, height])

        # Hightlight Start Node
        plt.scatter([self.start.x * self.grid_resolution],
                    [self.start.y * self.grid_resolution], c="green", s=100, alpha=1.0)

        # Hightlight Start Node
        plt.scatter([self.goal.x * self.grid_resolution],
                    [self.goal.y * self.grid_resolution], c="red", s=100, alpha=1.0)

        # Plot all edges in the graph
        if visualize_nodes:
            for node in self.node_list:
                if node.parent is not None:
                    plt.plot([node.x * self.grid_resolution,
                              node.parent.x * self.grid_resolution],
                              [node.y * self.grid_resolution,
                               node.parent.y * self.grid_resolution],
                             color="blue", alpha=0.5)

            # Plot the nodes
            plt.scatter([n.x * self.grid_resolution for n in self.node_list],
                        [n.y * self.grid_resolution for n in self.node_list],
                         c="red", s=10, alpha=0.5)

        # Set the ticks for the principal axes to denote the grid
        plt.xticks(np.arange(0, width, self.grid_resolution), minor=True)
        plt.yticks(np.arange(0, height, self.grid_resolution), minor=True)
```

```
        plt.grid(which='minor', alpha=0.1)

        plt.title(f"RRT* Graph: {len(self.node_list)} Nodes")
        plt.xlabel("X")
        plt.ylabel("Y")

        plt.show()

if __name__ == "__main__":

    # Load Grid
    # grid = np.loadtxt("grid_1.txt", delimiter=",")
    grid = np.loadtxt("grid_2.txt", delimiter=",")

    # Create RRT* object
    rrt_star = RRTStar(start=(5, 5),
                       goal=(110, 110),
                       obstacle_grid=grid,
                       expand_distance=3,
                       connect_circle_dist=5)
    # Plan
    rrt_star.plan(100)

    # Plot
    rrt_star.plot(visualize_nodes=True)
```

- A `Node` class has been provided for you to easily visualize the output. The nodes are connected by their parent members.
- Please fill out the functions as best as you can and then use them to your advantage in building the graph. At the end, you are to complete the RRT* algorithm in the `plan` function and it should end up looking like a bunch of function calls with the given parameters.
- Please add the new nodes for the algorithm into the `self.node_list` member, that way they can be visualized using the `plot` function. An example visualization is the following:



RRT* Pseudo Code

The pseudo-code for RRT* is the following:

$V \leftarrow \{x_{\text{init}}\}$; $E \leftarrow \emptyset$;
for $i = 1, \ldots, n$ **do**
  $x_{\text{rand}} \leftarrow \texttt{SampleFree}_i$;
  $x_{\text{nearest}} \leftarrow \texttt{Nearest}(G = (V, E), x_{\text{rand}})$;
  $x_{\text{new}} \leftarrow \texttt{Steer}(x_{\text{nearest}}, x_{\text{rand}})$;
  if $\texttt{ObtacleFree}(x_{\text{nearest}}, x_{\text{new}})$ **then**
    $X_{\text{near}} \leftarrow \texttt{Near}(G = (V, E), x_{\text{new}}, \min\{\gamma_{\text{RRT}^*}(\log(\text{card}\,(V))/\,\text{card}\,(V))^{1/d}, \eta\})$;
    $V \leftarrow V \cup \{x_{\text{new}}\}$;
    $x_{\text{min}} \leftarrow x_{\text{nearest}}$; $c_{\text{min}} \leftarrow \texttt{Cost}(x_{\text{nearest}}) + c(\texttt{Line}(x_{\text{nearest}}, x_{\text{new}}))$;
    foreach $x_{\text{near}} \in X_{\text{near}}$ **do** // Connect along a minimum-cost path
      if $\texttt{CollisionFree}(x_{\text{near}}, x_{\text{new}}) \wedge \texttt{Cost}(x_{\text{near}}) + c(\texttt{Line}(x_{\text{near}}, x_{\text{new}})) < c_{\text{min}}$ **then**
        $x_{\text{min}} \leftarrow x_{\text{near}}$; $c_{\text{min}} \leftarrow \texttt{Cost}(x_{\text{near}}) + c(\texttt{Line}(x_{\text{near}}, x_{\text{new}}))$
    $E \leftarrow E \cup \{(x_{\text{min}}, x_{\text{new}})\}$;
    foreach $x_{\text{near}} \in X_{\text{near}}$ **do** // Rewire the tree
      if $\texttt{CollisionFree}(x_{\text{new}}, x_{\text{near}}) \wedge \texttt{Cost}(x_{\text{new}}) + c(\texttt{Line}(x_{\text{new}}, x_{\text{near}})) < \texttt{Cost}(x_{\text{near}})$
      then $x_{\text{parent}} \leftarrow \texttt{Parent}(x_{\text{near}})$;
      $E \leftarrow (E \setminus \{(x_{\text{parent}}, x_{\text{near}})\}) \cup \{(x_{\text{new}}, x_{\text{near}})\}$
return $G = (V, E)$;

- As mentioned before, the main difference between RRT and RRT* is the connectivity to the parent nodes and the “rewiring” which optimizes the graph structure for finding the paths between the nodes. It essentially looks for lower cost nodes from the parent to the child node, which ends up providing shorter paths. More iterations **ideally** should provide shirter paths.
- Apart from the start and goal configurations, feel free to choose the rest of the parameters according to your implementation.

Grid Maps

To sample points from the space, please use the provided maps `grid_1.txt` **(https://elearn.ucr.edu/courses/215657/files/23980286?wrap=1)** **(https://elearn.ucr.edu/courses/215657/files/23980286/download?download_frd=1)** and `grid_2.txt` **(https://elearn.ucr.edu/courses/215657/files/23980287?wrap=1)** **(https://elearn.ucr.edu/courses/215657/files/23980287/download?download_frd=1)** . The resolution for both the grids is 0.25 m, the only difference between them is the size. `grid_1.txt` is a 10 m X 10 m grid and `grid_2.txt` is a 30 m X 30 m grid. For `grid_1.txt` , please set `start` to `(5, 5)` and goal to `(10, 10)` . For `grid_2.txt` , please set `start` to `(5, 5)` and goal to `(110, 110)` .

Collision Checking

For collision checking between 2 points, you can use Bresenham's Line Algorithm. The line algorithm itself approximates a line segment, as an ordered list of 2D pixels. You can use the algorithm to return a list of 2D points and you can iterate over the list to determine if the value in the grid is an obstacle or not. The pseudo-code for Bresenham's Line algorithm is the following:

```
function line(x0, y0, x1, y1)
   dx := abs(x1-x0)
   dy := abs(y1-y0)
   if x0 < x1 then sx := 1 else sx := -1
   if y0 < y1 then sy := 1 else sy := -1
   err := dx-dy

   loop
     setPixel(x0,y0)
     if x0 = x1 and y0 = y1 exit loop
     e2 := 2*err
     if e2 > -dy then
       err := err - dy
       x0 := x0 + sx
     end if
     if e2 <  dx then
       err := err + dx
       y0 := y0 + sy
     end if
   end loop
```

You can use the provided `Node` structures `x` and `y` components and `(x0, y0)` and `(x1, y1)`. The obstacle values in the cell will have a value of `1` and the free space will be represented as a `0` .

Submission

1. Submission: submission via Canvas / eLearn
2. Demo: not required
3. Due time: 11:59pm, March 11, Thursday
4. Files to submit:
   - lab6_report.pdf
   - rrt_star.py
5. Grading rubric:
   - 30% Clearly describe your approach and explain your code in the lab report.
   - 80% Implement point RRT* algorithm and pass all test cases.
   - 15% Penalty applies for each late day.

Additional Resources

- Bresenham's Line Algorithm - **Wiki** (https://en.wikipedia.org/wiki/Bresenham%27s_line_algorithm) , **Columbia University Slides** (https://www.cs.columbia.edu/~sedwards/classes/2015/4840/lines.pdf)
- RRT* Algorithm: **Karaman and Frazzoli** (https://arxiv.org/pdf/1105.1186) , **Lecture 11,** (https://elearn.ucr.edu/courses/215657/files/23880516?wrap=1) (https://elearn.ucr.edu/courses/215657/files/23880516/download?download_frd=1) **Berkeley Motion Planning Lecture Slides** (https://people.eecs.berkeley.edu/~pabbeel/cs287-fa19/slides/Lec10-motion-planning.pdf)